HW 4.3 Reproducible Analysis

Mohammed Alghamdi

2026-04-13

Busiest Airports Analysis

Busiest Airports Table

Table 1: Passenger traffic for six major airports from 2020 to 2025.

Airport	IATA	2020	2021	2022	2023	2024	2025
Beijing Daxing International Airport	PKX	NA	25051012	NA	NA	49441029	53,618,949
Dubai International Airport	DXB	25,836,771	29110609	66069981	86994365	92331506	95,200,000
Frankfurt Airport	FRA	18,768,601	24812849	48918482	59355389	61564957	63,189,666
Los Angeles International Airport	LAX	28,779,527	48007284	65924298	75050875	76588028	73,709,594
Tokyo Haneda Airport	HND	31,055,210	25876429	50290705	78719302	85900167	91,430,000

Busiest Airports Plot

Table Table 1 contains data on the number of passengers from six big airports in the years between 2020 and 2025. This table can be very helpful as it provides the actual figures for each airport. It also allows comparison for the same airport over several years. Graph Figure 1 facilitates the analysis of the trends by making them more visible. It emphasizes the change in the amount of passengers and their similarity/differences in recovery following low traffic periods.

Monte Carlo Numerical Integration

Small Multiple Visualization

Monte Carlo integration estimates area using random sampling. As the number of points increases, the estimate becomes more stable and closer to the true value.

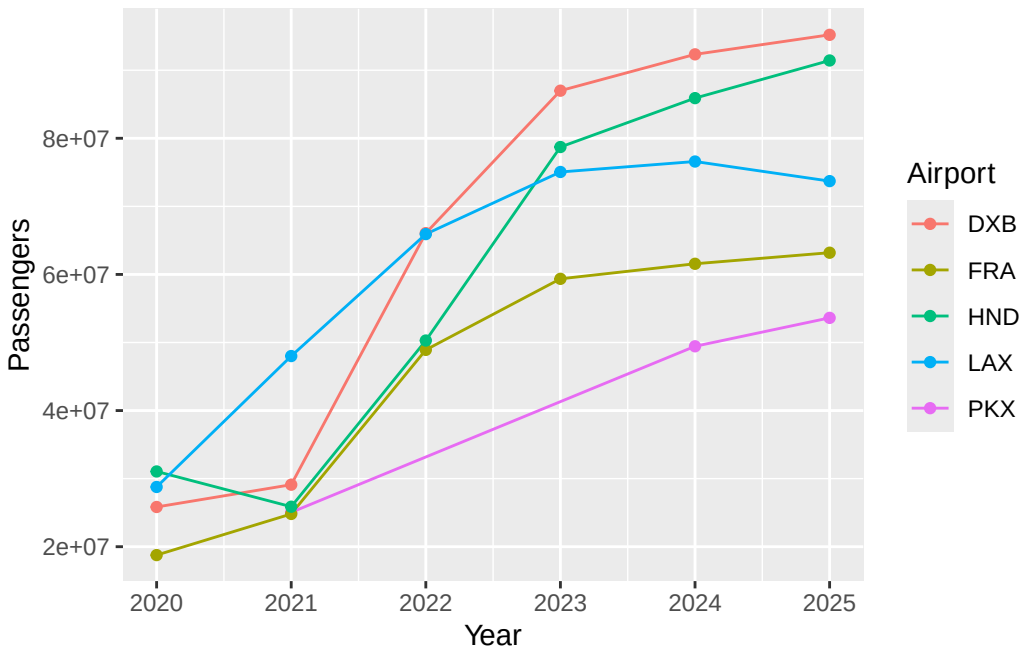
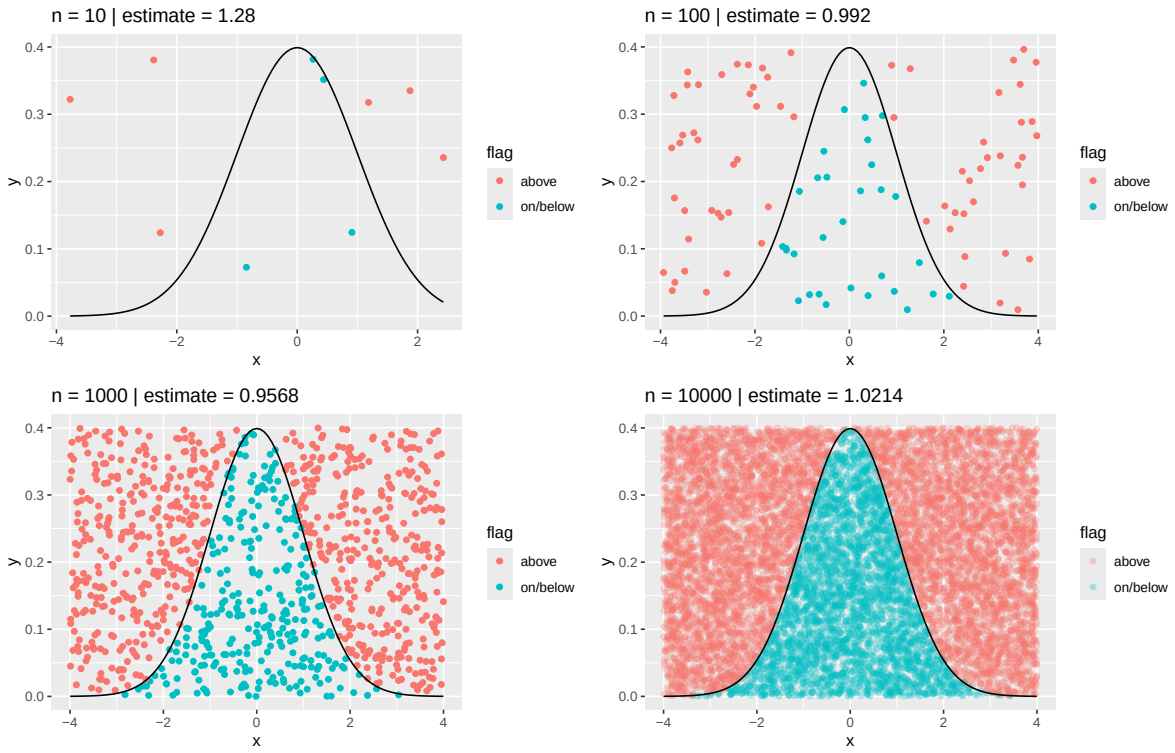


Figure 1: Passenger traffic trends for six major airports from 2020 to 2025.



Planning and Prompting GenAI Tools

Data Wrangling Plan

The goal is to clean the data and make a clear graph.

Steps:

1. Load the data into R.
2. Look at the data to understand it.
3. Clean the column names.
4. Check for missing values.
5. Fix missing values.
6. Make sure columns have correct types.
7. Create new columns if needed.
8. Arrange or sort the data.
9. Group the data.
10. Summarize the data.
11. Make the data tidy.
12. Create a graph.
13. Add labels and titles.
14. Check the final result.

Plan-Based Prompt Result

I used Anthropic Claude and gave it my detailed plan along with the dataset.

Claude cleaned the data, reshaped it into a tidy format, and created a clear visualization comparing the two groups over time.

The output included: - A cleaned dataset - A structured approach to tidying the data - A clear visualization of trends across time

The results showed that both groups remained stable at first and then decreased at the final measurement. The trends were easy to understand and clearly presented.

GenAI Response (Claude)

Ran 3 commands, viewed a file, read a file

Here's a full walkthrough of every step:

Step 1 — Load: 16 rows × 8 columns, all numeric (float64). The first 4 columns were the Treatment group, the last 4 were Control.

Step 2 — Rename columns: Original names (y0, y1, y2, y3, y0.1…) were unclear, so they were renamed to treat_base, treat_y1, treat_y2, treat_y3, ctrl_base, ctrl_y1, ctrl_y2, ctrl_y3.

Step 3 — Missing values: Row 15 in the Treatment group had all 4 values missing (NaN). The Control group had no missing values.

Step 4 — Fix missing values: Filled the 4 missing Treatment values with their respective column means — a standard imputation approach that doesn't distort the averages.

Step 5 — Types: All columns were already float64 — no conversion needed.

Step 6–8 — Reshape & group: The wide table was melted into a tidy long format with three columns: Time, Calcium, and Group. This makes it easy to compute group means and plot.

Step 9 — Summarize (means):

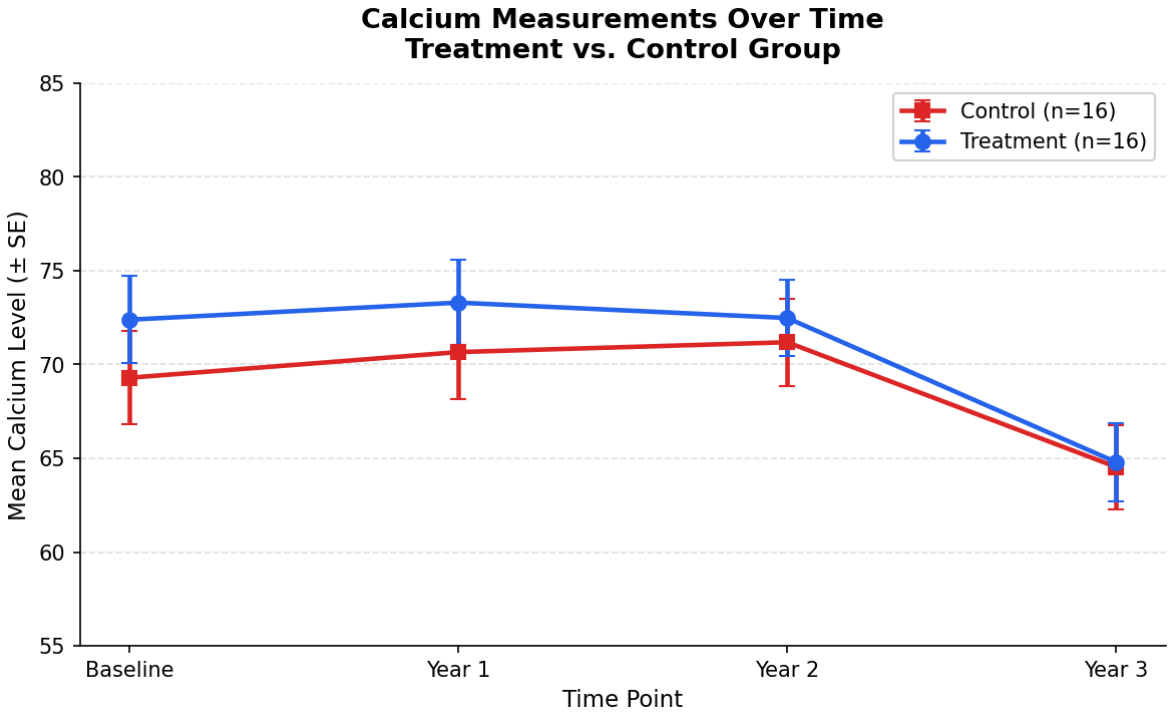
Treatment: 72.4, 73.3, 72.5, 64.8
Control: 69.3, 70.7, 71.2, 64.5

What the plot tells us in plain words:

For both groups, the initial calcium level was almost the same (~69-72). There wasn't much change in calcium levels during Baseline up until Year 2 for both groups. However, there is a noticeable decline in calcium for both groups at Year 3, where the levels went down to about 64-65. The Treatment group had higher calcium levels compared to the Control, although the trends were almost the same for both.

There is overlap in the error bars, indicating that the differences between both groups were insignificant.

Visualization from Claude



The graph below illustrates the changes in average levels of calcium over time between the treatment group and the control group. Both groups have almost equal levels initially, and the levels stay relatively constant until Year 2, after which there is a drop off in both cases. It can be seen that the treatment group is always just above the control group.

Generic Prompt Result

In the graph presented below, one can find out about the trends of average calcium levels between two groups – the treatment group and the control group. They have quite close levels at the beginning, and they remain fairly constant until the Year 2, when both of them start to decline. However, it is possible to note that throughout the process the treatment group's level stays slightly higher than the control group's.

The generic prompt provided a similar kind of visualization; however, there was no particular step-by-step approach followed while explaining the results.

On the whole, the generic prompt worked but not as efficiently as the plan-based one.

Comparison

The plan-driven prompt aided in generating an organized response. Claude was able to strictly adhere to the prompts by cleansing the data, managing the missing values, reconfiguring the dataset, and summarizing the outcomes. The output generated from the prompt was easily comprehensible.

The use of a plan-based prompt is better than that of a generic prompt since a plan gives guidance and structure on what to do next. While a generic response will still be able to give the desired result, it may lack accuracy or be unorganized.

In summary, utilizing plans helps generate better responses since they guide the GenAI tool through the process.

Reflection

Throughout STAT 184, I've been introduced to various ways of working with data utilizing data verbs and viewing data as an object that can be processed at each step of the way. Rather than writing elaborate lines of code, I focused on learning to use simple and straightforward functions like filter(), select(), mutate(), and summarise() to execute certain operations on a dataset.

The notion of having cases represent each row in the table was one of the core themes that I have come across in STAT 184. To achieve this, it's possible to use pivot_wider() or other similar functions in R. Moreover, I have been introduced to such concepts as group_by() where data gets sorted, followed by summarise() and count().

Chaining commands became one of the techniques used in this course to create efficient code using the pipe operator (|>) which allows performing actions sequentially and, therefore, improves readability.

Furthermore, throughout the course, I have been practicing creating tables and other forms of outputting data. I have learned how to use the kable() function to generate tables that are visually appealing and easy for readers to perceive.

Lastly, another valuable concept learned during STAT 184 classes is the understanding of the principle of simulation based on the Monte Carlo method. By increasing the sample size used in the simulations, I managed to see more precise estimates being produced.

GenAI Usage Appendix

Tool Used: Anthropic Claude
Version: Latest available version (Claude)
Date Used: April 15, 2026

Prompt Used: I uploaded the Calcium.csv dataset and asked Claude to follow my data wrangling plan step by step. The plan included cleaning the data, handling missing values, reshaping the dataset into a tidy format, summarizing the results, and creating a visualization comparing treatment and control groups over time.

Response: Ran 3 commands, viewed a file, read a file

GenAI Response (Claude)

Ran 3 commands, viewed a file, read a file

Here's a full walkthrough of every step:

Step 1 — Load: 16 rows × 8 columns, all numeric (float64). The first 4 columns were the Treatment group, the last 4 were Control.

Step 2 — Rename columns: Original names (y0, y1, y2, y3, y0.1…) were unclear, so they were renamed to treat_base, treat_y1, treat_y2, treat_y3, ctrl_base, ctrl_y1, ctrl_y2, ctrl_y3.

Step 3 — Missing values: Row 15 in the Treatment group had all 4 values missing (NaN). The Control group had no missing values.

Step 4 — Fix missing values: Filled the 4 missing Treatment values with their respective column means — a standard imputation approach that doesn't distort the averages.

Step 5 — Types: All columns were already float64 — no conversion needed.

Step 6–8 — Reshape & group: The wide table was melted into a tidy long format with three columns: Time, Calcium, and Group. This makes it easy to compute group means and plot.

Step 9 — Summarize (means):

Treatment: 72.4, 73.3, 72.5, 64.8
Control: 69.3, 70.7, 71.2, 64.5

What the plot tells us in plain words:

Both groups started with similar calcium levels at baseline (69–72).
Calcium stayed relatively stable from Baseline through Year 2 in both groups.
Then both groups show a notable drop at Year 3, falling to around 64–65.
The Treatment group was consistently slightly higher than Control throughout, but the two groups follow nearly identical trends suggesting the treatment had minimal effect on the overall calcium trajectory.

The error bars overlap at all time points, meaning the difference between groups is likely not statistically significant.

Code Appendix

All code used in this analysis is included in the Quarto document but hidden from the main report. The code was used to:

- Load and clean the datasets
- Create summary tables
- Generate visualizations
- Perform Monte Carlo simulation

The code is properly formatted and commented within the Quarto file.